

BRD

BRD

Business Problem Statement :

- To Streamline Python HR's Workflow to reduce interaction with multiple Google Sheets.

What is being proposed

- Search Candidates
- Update Candidate details
- Send Forms through email
- Update Payment Data
- Send Contact mails to candidates

Project Scope

- Zero Budget
- No Cloud (Paid services)
- Web (Primary) - Mobile Responsive
- No Mobile App
- No Attendance and Salary Entry as its already in Tickora
- No Detailed Student details thats already in Zoho
- No Chat/Messaging/Notifications as it's already in Whatsapp

Business Objectives:

- Centralize student management
- Create a single platform to manage all students information in one place
- Reduce manual work and dependency on multiple google sheets

Stakeholders:

- Python HR
- Branch Managers (Branch/Online Entry Form)
- Candidates (New and BGV form)
- Other HRs

Proposed Story Flow

She opens the app, gets mail id from candidates and enters it in a push panel, which sends either forms for new registrations, or for BGV, or shares mail ids for contacts. If a student got a form sent, here a notif is added to the track list as to show form pending for that candidate. The candidate would have filled the form which has stored the information in a google sheet. And this data of candidates has created a new profile for that student which can be seen in our application and we can update or Modify any data which also reflects in the google sheet, we can also add payment related updates and follow ups which are getting stored in their respective sheets. She can also use the dynamic search panel which automatically filters candidates as she types with the filters to retrieve the candidates information and populate it on screen. And for candidates who got sent BGV form, in their BGV status it shows form pending first. After the form gets

filled by the candidate it shows an option to review and send which automatically mails to the BGV team. Almost all routine details can be configured on screen with edit (pencil button). And every payment update is logged in a google sheet, but only the updates or modifications are reflected in the page. To fully support the student lifecycle, the system tracks finances across four distinct, independent pipeline buckets: Registration Fee for the upfront commitment, Course Fee for core tuition, Document Fee for materials and admin costs, and Placement Payment for the success fee charged after landing a job. Because prices can change due to future discounts, scholarships, or fee re-negotiations, she can use the built-in Adjustment Engine to apply these changes directly to any of the four buckets at any point in time. The app then runs smart "Net Payable" math on the fly, instantly calculating Base Fee minus Applied Discount to update the Net Payable Balance, automatically dropping the pending due amount on screen the second a discount tag is applied so she never has to do manual subtraction. And she can pull up a upgraded "changes" tab sidebar on the side in the payment panel to view all payment entries/corrections/changes made for that particular candidate, which now acts as a "Why" log; whenever she processes a structural fee override, the system forces her to log the reason, creating an unalterable audit trail for the business owners. Also clicking on the candidate name on any panel brings up the expanded candidate profile workspace which organizes complex career data into a clean, intuitive layout. Within this upgraded profile architecture, she can access a streamlined Past Employment Ledger that uses a simple, ergonomic text array to quickly type or view past company names like TCS, Cognizant, or Infosys, providing instant context without any data bloat. Next to this is the Interactive Document Vault, a structural powerhouse that splits candidate documentation into two organized visual columns: Documents Received and Documents Applied. To make tracking lightning-fast, this vault uses a preset checklist for standard employment documents like Offer Letters, Appraisals, Payslips, Relieving Letters, and Counter Offers, allowing her to toggle these presets with a single click to see exactly what has been submitted versus what is still outstanding. Finally, the core status indicators sitting right at the top of the profile act as highly mutable living tags that can be changed at a moment's notice using the global Pencil Edit tool, allowing her to smoothly toggle the BGV Status as checks progress, switch the Placed Tag to mark a student's transition to a successful graduate, and instantly update the dynamic Placed Company Tag once the hiring deal is finalized.

Current State Analysis

- Collects data from whatsapp
- gets inputs from Forms and Students
- Manually Enters data in Google Sheets
- Checks mail for BGV status
- Mail manually for BGV initiation
- Manually makes entry to whatsapp and Zoho
- Uploads Student attendance batchwise to whatsapp

- Manually gets Inspection details and puts in whatsapp

Future State Vision

A Single Web Application with Ergonomic design that is both Mobile and Laptop Responsive for easy CRUD of Data respective to the role of the Python HR.

Functional Requirements

Functional Requirements Documentation (FRD)

Project Title: Unified Candidate Management CRM (The Command Center)

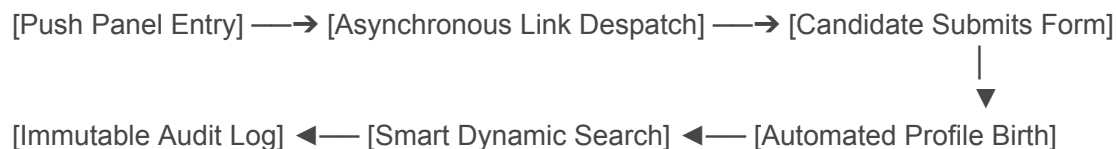
Target User: HR Team, Branch Managers, Executive Stakeholders

Core Backend Architecture: Google Sheets (Relational Tabs) via Web API Connection

1. Document Overview & System Workflow

This document specifies the official functional requirements for the Command Center & Push Panel web application. The platform acts as an administrative shell over a Google Sheets backend, designed to streamline candidate onboarding, background verification (BGV), documentation, lifecycle tracking, and financial reconciliation while preventing data corruption or accidental manual deletion.

System Flow Diagram



2. System Modules & Functional Requirements

2.1 Module 1: The Command Center & Push Panel

The landing interface provides a responsive dashboard engineered for single-keystroke operations on desktop and mobile platforms.

- **FR-1.1: Payload Dispatcher**
 - The system **shall** feature a single input container (The Push Panel) optimized to accept raw text strings (primarily single email addresses).
 - The system **must** provide three distinct payload toggles: New Registration Form, BGV Form, and Contact Mail.
- **FR-1.2: Asynchronous Delivery Execution**
 - Upon hitting the Send trigger, the application **shall** process email dispatching in a background thread to prevent UI locking.
- **FR-1.3: Initialization State**
 - The platform **shall** instantly create an entry in the tracked list with a temporary milestone flag set to Form Pending.

2.2 Module 2: Automated Profile Birth

Eliminates manual typing by making the candidate the primary point of data ingestion.

- **FR-2.1: Mobile-Optimized Data Intake Form**
 - The links sent to candidates **must** resolve to an independent web view styled for high mobile usability.
- **FR-2.2: Dual-Write Ingestion Pipeline**
 - Upon submission, data **shall** instantly execute a raw append to the underlying target Google Sheet row and simultaneously push a state-change webhook to the frontend.
- **FR-2.3: Zero-Latency UI Re-rendering**
 - The active interface for HR **shall** automatically remove the **Form Pending** state and instantiate a fully populated **Candidate Profile Workspace** without requiring a webpage refresh.

2.3 Module 3: Dynamic Search Panel & Inline Editor

Enables precision lookup using fuzzy logic and acts as a security barrier for modifying baseline student records.

```
+-----+
| 🔍 Search Input Bar (Accepts Name, Email, or Normalized Numbers) |
+-----+
| [⌚ Form Pending ×] [🛡️ BGV: Cleared ×] [💰 Has Dues ×] [+ Add Matrix Filter] |
+-----+
```

- **FR-3.1: Multi-Modal Normalized Matching Engine**
 - The search engine **shall** evaluate inputs against name, email strings, and phone number parameters simultaneously.
 - The system **must** run an input sanitization script that strips non-numeric characters (+, -, spaces) from phone strings using regex: `[^\d]`
 - Matches **must** update the visible UI results array dynamically with every keystroke (real-time filtering).
- **FR-3.2: Inline Modification Shield (The Pencil Editor)**
 - Baseline profile data fields **shall** default to a read-only state.
 - An explicit tap on the inline **Pencil Icon** **shall** convert the field to an editable text box or dropdown field.
 - Modifications committed via the Pencil Editor **must** execute a precise background cell update on the target Google Sheet without risking full-row or column deletions.

2.4 Module 4: Expanded Candidate Profile Workspace

Acts as the central operational cockpit for managing a candidate's ongoing compliance and placement metrics.

+-----+
| MASTER BADGES: BGV Status: [Pending ] Placed: [No ] Company: [-- ] |
+-----+

- **FR-4.1: Top-Level Living Tags**

- The platform **must** feature persistent status badges displaying BGV Status, Placed Tag, and Placed Company Tag at the absolute top of the profile matrix.
- Modifying these tags using the inline editor **shall** instantly flag updates out to downstream entities (Branch Managers, Placement Officers).

- **FR-4.2: Past Employment Ledger (Array Tag Format)**

- Instead of complex sub-tables, previous employment history **shall** be managed via an interactive text array wrapper.
- HR **shall** input string entries (e.g., "TCS"), press a confirmation key, and see the system bind it as a discrete visual chip.
- The application **shall** compress this data array into a single comma-separated text line string when pushing structural records back down to the target spreadsheet cell.

- **FR-4.3: Interactive Preset Document Matrix**

- The documentation sub-view **shall** be segmented into two structural buckets: Documents Received and Documents Applied.
- The interface **must** display an immutable checklist consisting of five hardcoded items:
 1.  Offer Letter
 2.  Appraisals
 3.  Payslips
 4.  Relieving Letter
 5.  Counter Offer
- Checking a preset box **shall** move that file category into the active state in real time.

2.5 Module 5: Financial Management & Adjustment Engine

This module treats student payments with the operational rigor of an ERP, splitting items into sequential modules and calculating figures natively in the web wrapper.

The 4 Independent Financial Pipelines

1. **Registration Fee:** Upfront seat booking/commitment collection.
2. **Course Fee:** Base tuition core allocation.
3. **Document Fee:** Examination tokens, books, and compliance fees.
4. **Placement Payment:** Success fee triggered upon successful hire confirmation.

+-----+	
 FINANCIAL MANAGEMENT INTERFACE	
[Registration Tab] [ Course Fee Tab] [Document Tab] [Placement Tab]	
+-----+	
Base Fee Amount: \$1,000.00	
Applied Discounts / Scholarship Adjustments: -\$200.00 (Corporate Waiver )	
+-----+	
Net Payable Balance: \$800.00	
Paid To Date: \$500.00	
Net Remaining Pending Dues: \$300.00	
+-----+	

- **FR-5.1: Targeted Adjustment Engine**
 - HR **shall** be permitted to inject structural adjustments (discounts, structural write-offs, scholarship overrides) directly into any of the 4 individual pipelines independently.
- **FR-5.2: Automated Balance Calculation Architecture**
 - The web app layer **must** automatically process financial balance tracking dynamically using the following mathematical formulas:

Net Payable Balance = Base Fee – Applied Discount

Pending Due = Net Payable Balance – Paid to Date

- The interface **shall** automatically update and drop the visible Pending Due value the exact second an adjustment tag is written to the interface.

2.6 Module 6: BGV Hand-Off Pipeline & The Changes Sidebar

- **FR-6.1: State-Driven Automation Trigger**
 - The moment a candidate's custom BGV intake web form is submitted, their global row indicator **shall** automatically transition from Form Pending to an active Review and Send call-to-action button on HR's dashboard.
- **FR-6.2: One-Click Third-Party BGV Provisioning**
 - Clicking Review and Send **shall** present a summary verification window. Confirming this step **shall** cause the web app to build a clean JSON data packet

and auto-dispatch a structured BGV notification directly to the external vetting agency's email queue.

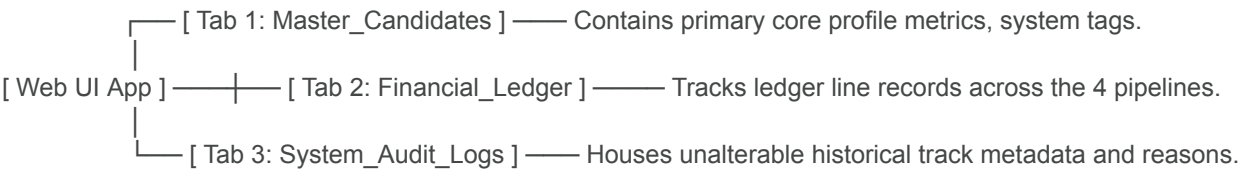
- **FR-6.3: Immutable Operational Overwrite Audit Log**
 - The panel **shall** include a slide-out drawer (The Changes Sidebar) displaying log details categorized by transaction type:

Log Categorization Type	Required Data Metadata Captured
 Financial Log Entry	Captures payment values, date stamps, receipt tracking codes, and transactional confirmations.
 Structural Overwrite Log	Captures shifts in raw base fees, applied discount logic, and manual flag changes.

- **FR-6.4: Enforced Accountability Constraints**
 - If an administrative user modifies a baseline payment parameters field or applies an adjustment waiver, the **Commit Save** function **shall remain locked** until a comprehensive narrative entry is entered into the mandatory text field box (e.g., *"Branch Manager approved early-bird corporate waiver"*).
 - This narrative data line **shall** be committed to an un-editable audit ledger sheet alongside a user stamp.

3. Database Schema Strategy (Google Sheets Routing)

To prevent data corruption and maintain processing speeds, the system **shall** divide information across a multi-tab structural pattern inside the target workbook workbook:



- **Data Isolation Rule:** The client interface **shall** execute reads and writes via index matches using a unique, immutable candidate structural identifier mapping across all three spreadsheet tabs simultaneously.